

Observability & Chaos Engineering - 2

EMPOWERING High Performance
Technology Teams

Agenda

Duration	Activity
15 mins	Welcome and Objectives Brief Recap of observability and chaos engineering concepts
20 mins	UseCase Discussion 1 High CPU Spikes in an Auto-Scaling Web Application
20 mins	UseCase Discussion 2 Chaos Experiment – Inject CPU Stress and Observe
15 mins	Break
50 mins	Hands on Implementation and Concept Discussion
10 mins	Scenario Based Question Task
10 mins	Summary Key Takeaways and Case Study Discussion
10 mins	Q&A

Note: For sessions that are completely demo/Lab or notebook based, please keep one agenda slide and explain the same thing in the beginning so that participants are aware they do not have PPT.

Introduction to Observability & Chaos Engineering

- **Observability:**
 - Measures system health and performance
 - Includes Logs, Metrics, Traces
 - Tools: ?
- **Chaos Engineering:**
 - Testing system resilience by injecting failures
 - Tools: Manual Stress, AWS Fault Injection Simulator
 - Goal: ?

UseCase 1: High CPU Spikes in an Auto-Scaling Web Application

Scenario:

A microservices-based e-commerce site experiences **random CPU spikes** during product launches.
CPU-intensive processes (e.g., image processing, checkout) overload certain EC2 instances.

Challenges:

No proactive **alerting on CPU utilization**.
Auto-scaling fails because spikes are unpredictable.

Implementation:

Deploy **EC2 instances with CloudWatch Agent**.
Configure **metric collection and anomaly detection** for CPU utilization.

UseCase 2: Chaos Experiment – Inject CPU Stress and Observe

Scenario:

A fintech company wants to ensure **backend payment services** remain operational under high CPU stress.

Implementation:

Use **AWS Fault Injection Simulator (FIS)** or manually inject CPU load.

Observe how CloudWatch detects anomalies and triggers alerts.

Expected Outcome:

Learn to **correlate CPU anomalies with system performance**.

Understand how **anomaly detection differentiates normal spikes vs. abnormal behavior**.

Hands on Implementation

Hands-on Lab

- **Implementing Anomaly Detection for EC2 CPU Spikes using Terraform Automation Script**

- ✓ Launch an EC2 instance and install CloudWatch Agent.
- ✓ Configure CloudWatch metrics for CPU utilization.
- ✓ Enable Anomaly Detection for high CPU spikes.
- ✓ Simulate CPU stress using a load generator.
- ✓ Observe and analyze metric anomalies in CloudWatch Dashboard.

Observability and Chaos Engineering

- [CTS-SRE-Ex1](#) using AWS service manually
- [Document.docx](#) Automation using Terraform
<https://github.com/AshishSharmaPrivate/ObservabilityChaosDemo/tree/main>

Scenario Based Question Task-1

Your company has EC2 instances running in **multiple AWS regions**, and you're monitoring the CPU utilization across all instances using CloudWatch etc. Recently, some regions are showing unusual CPU utilization spikes, but not all instances are affected. You need to monitor the CPU utilization across all instances and send alerts only when the spikes are **global** and affect the application as a whole !

Task:

- How would you **set up anomaly detection** to **track CPU utilization** across **multiple regions**?
- What method would you use to **aggregate metrics** from different regions to detect **global anomalies** (e.g., if a certain percentage of instances across regions show spikes)?
- How would you **set up alerting** so that only relevant spikes trigger notifications?

Scenario Based Question Task-2

During peak traffic periods (e.g., sales events, product launches), you know that CPU utilization will naturally spike across multiple EC2 instances. However, you still want to detect **unexpected spikes** caused by faulty code or configuration issues. You need to ensure that the system is only alerting on **abnormal** traffic, not just during high traffic periods !

Task:

- How would you **differentiate** between **expected high traffic spikes** and **unexpected CPU spikes**?
- What thresholds and **metrics** would you configure for anomaly detection during high traffic periods to ensure the system is only alerting on **unexpected anomalies**?
- How can you handle **bursty traffic spikes** without triggering alerts if the CPU utilization is above a certain threshold?

Summary

- Best practices for **monitoring and responding to CPU anomalies.**
- How **anomaly detection** helps prevent unplanned outages.
- Using **Chaos Engineering** to validate system resilience **Hands-on implementation.**
- **Task** (offline Assignment)

Thank You!
StackRoute
(An NIIT Venture)

